

ARIF 設計書

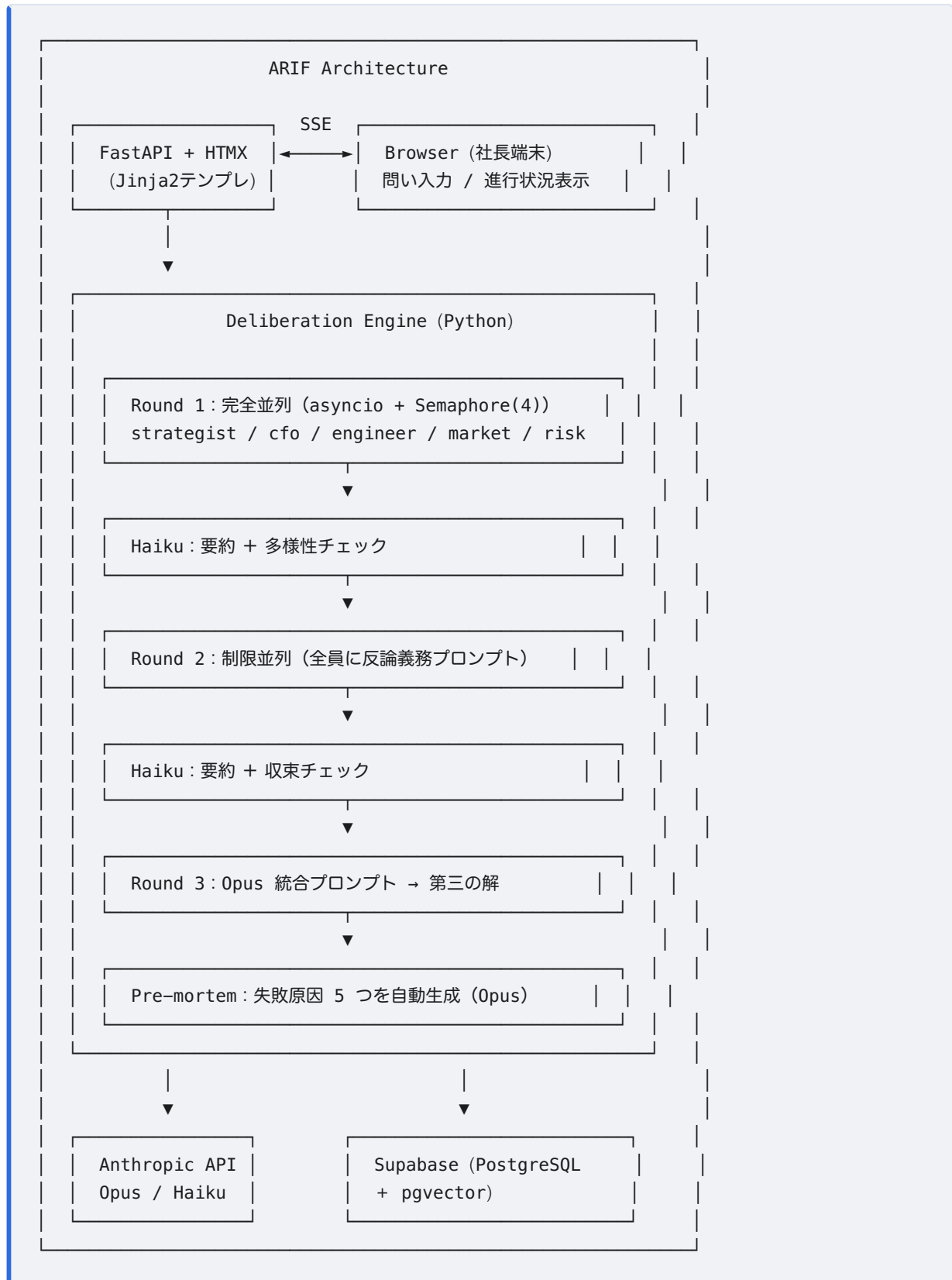
バージョン： 0.4

最終更新： 2026-05-26

ステータス： 設計諮問完了・上様確認待ち

1. アーキテクチャ概要

1.1 全体構成図



1.2 設計方針

方針	内容
フロントエンド	FastAPI + HTMX (Phase 3 まで)。ユーザーは社長 1 名のため SPA は過剰。
非同期処理	<code>asyncio</code> + <code>AsyncAnthropic</code> で Round 1・2 を並列化
コスト制御	Round 間要約を Haiku に委託し、Opus の入力トークンを削減
堅牢性	1 体の失敗で全体停止させない設計（エラー体はスキップして進行）
バージョン管理	モデルバージョンを <code>config.yaml</code> で一元ピン留め

2. モジュール構成

```
arif/
├── engine/
│   ├── agents.py      # エージェント定義・ペルソナ管理
│   ├── debate.py      # 3ラウンド熟議ループ（メインエンジン）
│   ├── memory.py      # Supabase 読み書き・RAG文脈生成
│   └── quality.py      # 多様性チェック・Pre-mortem収束判定
├── api/
│   ├── main.py        # FastAPI エントリーポイント・SSE
│   ├── routes/
│   │   ├── deliberations.py  # /deliberations エンドポイント
│   │   └── context.py        # /context エンドポイント
│   └── templates/          # Jinja2 + HTMX テンプレート
├── config.yaml           # モデルバージョン・定数の一元管理
├── tests/
│   ├── test_debate.py
│   ├── test_memory.py
│   └── test_quality.py
└── docs/
    ├── 仕様書.md
    └── 設計書.md（本文書）
```

3. 設定ファイル (config.yaml)

```

models:
  opus:      "claude-opus-4-20250514"      # 討論・統合・Pre-mortem
  haiku:     "claude-haiku-4-20250514"     # 要約・収束チェック・補助
  embedding: "text-embedding-3-small"      # pgvector 用ベクトル化

# —— モード定義（仕様書 F-009） ——
modes:
  speed:                                           # 早足
    rounds: 2
    debate_model: "haiku"
    summary_model: "haiku"
    synthesis_model: "haiku"
    extended_thinking: false
    pre_mortem: false
    estimated_seconds: 30
    cost_target_usd: 0.05

  standard:                                       # 常足（既定）
    rounds: 3
    debate_model: "opus"
    summary_model: "haiku"
    synthesis_model: "opus"
    extended_thinking: false
    pre_mortem: true
    estimated_seconds: 120
    cost_target_usd: 0.20

  deep:                                           # 熟考
    rounds: 3
    debate_model: "opus"
    summary_model: "opus"                       # 要約も Opus で精緻化
    synthesis_model: "opus"
    extended_thinking: true                     # 思考過程の可視化
    pre_mortem: true
    pre_mortem_variants: 3                      # 複数視点で事前検死
    background_recommended: true
    estimated_seconds: 300
    cost_target_usd: 0.50

# —— ストリーミング・段階開示（仕様書 F-010, F-011） ——
streaming:
  enabled: true                                 # Anthropic Streaming API 使用
  progressive_disclosure: true                 # 完了エージェントから順次表示
  first_token_target_ms: 1000                  # 初トークン到着目標時間

# —— プロンプトキャッシュ（仕様書 F-014） ——
caching:
  enable_prompt_caching: true
  cache_targets:                                # キャッシュ対象セクション
    - "agent_system_prompts"                  # エージェントのペルソナ（固定）
    - "company_context"                       # 会社基本情報（更新頻度低）

```

```

    - "principle_iv_template"          # 反論義務テンプレート
    ttl_minutes: 5                     # Anthropic 標準

# —— Extended Thinking (仕様書 F-012) ——
extended_thinking:
    enabled_by_default: false          # ユーザーが任意ON
    visible_to_user: true              # 思考過程を画面表示
    max_thinking_tokens: 8192

# —— 共通制限 ——
debate:
    concurrency_limit: 4
    max_tokens_per_agent: 2048
    timeout_seconds: 120
    max_api_calls_per_session: 25

# —— バックグラウンド・通知（仕様書 N-001~N-003） ——
background:
    enabled: true
    notification_channels:
        - "browser"                   # Web Push API
        - "email"                     # 任意
    result_url_template: "/deliberations/{session_id}"

```

4. エージェント設計 (agents.py)

4.1 データ構造

```

@dataclass
class Agent:
    id: str
    role: str
    system_prompt: str
    model: str
    temperature: float
    max_tokens: int = 2048
    optional: bool = False

```

4.2 温度パラメータの設計意図

エージェント	温度	意図
strategist	0.7	やや創造的。長期シナリオを多様に生成
cfo	0.3	保守的・数字重視。ブレない定量軸
engineer	0.4	現実的。実現可能性に集中
market	0.8	発散的。顧客視点の意外な気づきを促す
risk	0.2	最も保守的。最悪ケースを確実に拾う
philosopher	1.0	最も自由。常識を壊す問いを立てる
synthesizer	0.5	バランス型。統合と創造性を両立

温度パラメータを意図的に分散させることで、モデルが同一学習データを持っていても出力の多様性を構造的に確保する（群集効果防止の第一手段）。

5. 熟議フロー設計 (debate.py)

5.1 フロー詳細

入力: question (課題文字列)

↓

[前処理] memory.py で会社コンテキスト・過去議論を取得 (RAG)

↓

[Round 1] asyncio.gather で全エージェントを並列呼び出し
各エージェントは相互の意見を参照しない
失敗した体はスキップ (return_exceptions=True)

↓

[要約①] Haiku で Round 1 全意見を 1,500 tokens 以内に要約
多様性スコアを計算 (コサイン類似度の逆数)
多様性 < 0.3 の場合はログに警告

↓

[Round 2] asyncio.gather で全エージェントを並列呼び出し
入力: 要約された Round 1 意見 + 会社コンテキスト
プロンプト: 反論義務テンプレートを全員に適用

↓

[要約②] Haiku で Round 2 全意見を要約
収束チェック (全員の意見が一致していないか確認)

↓

[Round 3] Opus で統合プロンプトを1回呼び出し
入力: 要約① + 要約② + 「第三の解」品質基準
出力: 第三の解 (synthesis)

↓

[Pre-mortem] Opus で「この判断が失敗した場合の原因 5 つ」を生成

↓

[保存] Supabase に全ログ・コスト・レイテンシを永続化

↓

出力: DebateResult (synthesis + pre_mortem + 全ラウンドログ)

5.2 想定レイテンシ (モード別)

5.2.1 早足モード (speed)

ステップ	推定時間
Round 1 (5体Haiku並列)	~10秒
Round 2 (5体Haiku並列)	~15秒
統合 (Haiku)	~5秒

ステップ	推定時間
合計	約 30秒

5.2.2 常足モード（standard・既定）

ステップ	推定時間	キャッシュ時
RAG 文脈取得	~2秒	~1秒
Round 1（5体Opus並列）	~30秒	~15秒
Haiku 要約① + 多様性測定	~5秒	~5秒
Round 2（5体Opus並列）	~45秒	~25秒
Haiku 要約② + 説得依存度検定	~5秒	~5秒
Round 3 統合（Opus）	~30秒	~15秒
Pre-mortem（Opus）	~20秒	~10秒
合計	約 2分 17秒	約 1分 16秒

5.2.3 熟考モード（deep）

ステップ	推定時間
RAG 文脈取得（拡張）	~5秒
Round 1（5体Opus+Extended Thinking 並列）	~60秒
Opus 要約①	~10秒
Round 2（5体Opus+Extended Thinking 並列）	~90秒
Opus 要約②	~10秒
Round 3 統合	~45秒
Pre-mortem 3 視点	~60秒
合計	約 4分 40秒

体感時間の最小化策（モード共通）： - ストリーミング配信：初トークン到着 1 秒以内 - 段階的開示：完了エージェントから順次表示 - バックグラウンド実行+通知：熟考モードでは推奨

5.3 コスト試算（モード別）

5.3.1 常足モード（standard）

項目	トークン概算	通常コスト	キャッシュ時
Round 1（5体 × input ~3,000）	~15,000	~\$0.045	~\$0.018
Haiku 要約①	~5,000	~\$0.002	~\$0.001
Round 2（5体 × input ~8,000）	~40,000	~\$0.120	~\$0.060
Haiku 要約②	~3,000	~\$0.001	~\$0.001
Round 3 統合	~10,000	~\$0.030	~\$0.015
Pre-mortem	~5,000	~\$0.015	~\$0.008
合計	~78,000	~\$0.213	~\$0.103

目標 \$0.20 とほぼ同等。プロンプトキャッシュ有効時は約 半減。

5.3.2 早足モード（speed）

項目	コスト概算
全 Haiku ・ 2 ラウンド	~\$0.03

5.3.3 熟考モード（deep）

項目	コスト概算
全 Opus + Extended Thinking + Pre-mortem 3 視点	~\$0.45

6. データベース設計 (Supabase)

6.1 スキーマ

```

-- pgvector 拡張を有効化
CREATE EXTENSION IF NOT EXISTS vector;

-- =====
-- 熟議セッション
-- =====
CREATE TABLE deliberation_sessions (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    title             TEXT NOT NULL,
    question          TEXT NOT NULL,
    status            TEXT NOT NULL DEFAULT 'pending',
    -- pending / round1 / round2 / round3 / completed / failed
    config             JSONB DEFAULT '{}',

    synthesis         TEXT,
    pre_mortem        TEXT,
    executive_summary TEXT,

    total_input_tokens INTEGER DEFAULT 0,
    total_output_tokens INTEGER DEFAULT 0,
    total_cost_usd      DECIMAL(10,4) DEFAULT 0,
    duration_seconds   INTEGER,

    created_at        TIMESTAMPTZ DEFAULT NOW(),
    updated_at        TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- 各エージェントの発言ログ
-- =====
CREATE TABLE agent_responses (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id        UUID NOT NULL REFERENCES deliberation_sessions(id) ON DELETE
    CASCADE,

    round            INTEGER NOT NULL CHECK (round BETWEEN 1 AND 3),
    agent_id         TEXT NOT NULL,
    agent_role       TEXT NOT NULL,
    model_used       TEXT NOT NULL,
    temperature      DECIMAL(3,2),

    content          TEXT NOT NULL,

    -- Haiku で後処理した構造化データ
    key_points       JSONB DEFAULT '[]',
    stance           TEXT CHECK (stance IN ('support','oppose','neutral','conditional')),
    confidence       DECIMAL(3,2) CHECK (confidence BETWEEN 0 AND 1),

    -- コスト・パフォーマンス追跡
    input_tokens     INTEGER,
    output_tokens    INTEGER,

```

```

cost_usd          DECIMAL(10,6),
latency_ms        INTEGER,
error             TEXT,    -- 失敗した場合のエラーメッセージ

created_at        TIMESTAMPTZ DEFAULT NOW()
);

-- 1ラウンドに同一エージェントは1回のみ
CREATE UNIQUE INDEX idx_unique_response
  ON agent_responses(session_id, round, agent_id);

-- =====
-- 会社コンテキスト (RAG 用)
-- =====
CREATE TABLE company_context (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  category        TEXT NOT NULL,
  -- strategy / financial / org / market / risk / history
  title          TEXT NOT NULL,
  content         TEXT NOT NULL,
  embedding       vector(1536),
  source         TEXT,
  valid_from      DATE,
  valid_until     DATE,    -- NULL なら無期限
  created_at      TIMESTAMPTZ DEFAULT NOW(),
  updated_at      TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- セッション横断の洞察
-- =====
CREATE TABLE deliberation_insights (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id      UUID NOT NULL REFERENCES deliberation_sessions(id),
  insight_type    TEXT NOT NULL,
  -- decision / concern / pattern / principle
  content         TEXT NOT NULL,
  embedding       vector(1536),
  president_rating INTEGER CHECK (president_rating BETWEEN 1 AND 5),
  president_note  TEXT,
  is_active       BOOLEAN DEFAULT TRUE,
  created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- 社長フィードバック
-- =====
CREATE TABLE session_feedback (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id      UUID NOT NULL REFERENCES deliberation_sessions(id),
  overall_rating  INTEGER CHECK (overall_rating BETWEEN 1 AND 5),
  usefulness      INTEGER CHECK (usefulness BETWEEN 1 AND 5),

```

```

novelty          INTEGER CHECK (novelty BETWEEN 1 AND 5),
best_agent       TEXT,
worst_agent      TEXT,
free_comment     TEXT,
action_taken     TEXT,    -- 実際にどう意思決定したか（後日入力）
created_at      TIMESTAMPTZ DEFAULT NOW()
);

-- =====
-- pgvector インデックス
-- =====
CREATE INDEX idx_context_embedding ON company_context
    USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

CREATE INDEX idx_insights_embedding ON deliberation_insights
    USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);

```

6.2 テーブル関係図

```

deliberation_sessions
├── agent_responses (1セッション = 最大 15 レコード: 5体 × 3ラウンド)
├── deliberation_insights (セッションから抽出した洞察)
└── session_feedback (社長の事後評価)

company_context (セッションと独立。全セッションから参照)

```

7. API 設計

7.1 エンドポイント一覧

メソッド	パス	説明
GET	/health	ヘルスチェック
POST	/deliberations	熟議セッションを開始
GET	/deliberations	過去のセッション一覧
GET	/deliberations/{id}	セッション詳細（全ラウンドログ含む）
GET	/deliberations/{id}/stream	SSE による進行状況リアルタイム配信
POST	/deliberations/{id}/feedback	社長フィードバック登録
GET	/context	会社コンテキスト一覧
POST	/context	会社コンテキスト登録・更新

7.2 POST /deliberations

```
// Request
{
  "question": "当社の5年後の主力事業をどう描くべきか?",
  "title": "中期事業戦略2026",
  "mode": "standard",           // speed / standard / deep (既定: standard)
  "include_philosopher": false,
  "extended_thinking": false,  // 任意ON
  "background": false,         // バックグラウンド実行
  "notify_email": null         // 完了時メール通知 (任意)
}

// Response (即時返却。熟議はバックグラウンド実行)
{
  "session_id": "uuid",
  "status": "pending",
  "mode": "standard",
  "estimated_seconds": 120,
  "stream_url": "/deliberations/{uuid}/stream"
}
```

7.3 SSE イベント形式（ストリーミング対応版）

既存イベント

```
event: session_start
data: {"session_id": "uuid", "mode": "standard", "estimated_seconds": 120}

event: round_start
data: {"round": 1, "message": "Round 1 開始", "agents":
["strategist", "cfo", "engineer", "market", "risk"]}

event: agent_started
data: {"round": 1, "agent": "strategist", "started_at": "..."}

```

新規：トークン単位ストリーミング（仕様書 F-010）

```
event: agent_thinking_delta          # Extended Thinking 有効時のみ
data: {"agent": "strategist", "thinking_chunk": "中期事業戦略において..."}

event: agent_content_delta           # 通常応答のトークン配信
data: {"agent": "strategist", "text_chunk": "経営戦略として最重要なのは..."}

event: agent_done                    # 段階的開示（仕様書 F-011）
data: {"agent": "strategist", "full_content": "...", "tokens": 1850, "latency_ms":
18430}

```


ラウンド・統合イベント

```
event: round_summary
data: {"round": 1, "diversity_score_phi": 0.62, "convergence": false}

event: persuasion_check          # 説得依存度検定 (CFIM 原則II)
data: {"round": 2, "persuasion_risk": "low", "details": "..."}

event: synthesis_delta          # 統合もストリーミング
data: {"text_chunk": "第三の解として..." }

event: synthesis_done
data: {"synthesis": "...", "guilford_scores": {"fluency": 4, "flexibility": 5,
"originality": 4, "elaboration": 5}}

event: pre_mortem_done
data: {"failure_scenarios": ["...", "...", "...", "...", "..."]}

event: complete
data: {"session_id": "uuid", "cost_usd": 0.187, "duration_seconds": 142,
"cached_tokens": 12500}
```

バックグラウンドモード時の追加イベント

```
event: background_accepted
data: {"session_id": "uuid", "notify_channels": ["browser","email"]}
```

完了通知は別途 Web Push API・SMTP で送信。SSE 接続が切れていても通知される設計。

8. エラーハンドリング設計

エラーケース	対処
1体のAPI呼び出し失敗	スキップして他のエージェントで継続
Rate Limit (429)	指数バックオフリトライ (最大3回、待機 1s / 2s / 4s)
タイムアウト (120秒超)	そのエージェントをスキップ
コスト上限超過	セッションを中断し部分結果を返す
Supabase 接続失敗	セッション結果をメモリに保持し後続の保存をリトライ
全エージェント失敗	500エラーを返しセッションを failed 状態に更新

9. 品質管理設計 (quality.py)

本章の3機構は、CFIM白書 §4 原則IV「段階的検証」(Process Reward Model 拡張)の実装である。

Lightman ら (2024) の知見に従い、最終アウトカム評価のみならず各ラウンドのプロセス評価を行う。

9.1 多様性チェック (CFIM 干渉ポテンシャル Φ の測定)

Round 1 終了後、全エージェントの意見のコサイン類似度を計算し、多様性スコアを記録する。

多様性スコア = $1.0 - (\text{全ペアのコサイン類似度の平均}) \in [0, 2]$
スコア < 0.3: ログに警告 (プロンプト見直しの目安)

* CFIM白書 §3.3 の正規化干渉ポテンシャル $\Phi \in [0, 1]$ とは線形対応する。
閾値 0.3 (本書) $\Leftrightarrow \Phi < 0.15$ (白書)。

9.2 収束チェック・説得依存度検定

Round 2 終了後、Haiku により以下を判定する：

1. **全員一致検出**：全エージェントが同一結論に達していないか

2. **説得依存度検定**： Round 2 で後出のエージェントが Round 1 の特定エージェントに過度に類似していないか（Liu ら 2026 の persuasion-induced adversarial influence への対策）

いずれかが検出された場合、Round 3 のプロンプトに「構造的原因を分析せよ」を追加する。

9.3 Pre-mortem（事前検死）

Round 3 の統合後、以下のプロンプトで Opus を 1 回呼び出す。

以下の経営判断が1年後に大失敗していました。

[synthesis の内容]

失敗の原因として最もありえるものを5つ、具体的に列挙してください。

11. 速度最適化設計

本章は仕様書 §3.1b（F-009～F-014）および §5.1（モード別応答時間）の実装設計である。

11.1 ストリーミング配信 (F-010)

```
from anthropic import AsyncAnthropic

client = AsyncAnthropic()

async def stream_agent(agent: Agent, prompt: str, sse: SSEmitter):
    """エージェントの発言をトークン単位で SSE に流す"""
    async with client.messages.stream(
        model=agent.model,
        max_tokens=agent.max_tokens,
        temperature=agent.temperature,
        system=agent.system_prompt,
        messages=[{"role": "user", "content": prompt}],
    ) as stream:
        full_text = []
        async for event in stream:
            if event.type == "content_block_delta":
                chunk = event.delta.text
                full_text.append(chunk)
                await sse.send("agent_content_delta", {
                    "agent": agent.id,
                    "text_chunk": chunk,
                })
            elif event.type == "thinking_delta":
                await sse.send("agent_thinking_delta", {
                    "agent": agent.id,
                    "thinking_chunk": event.delta.thinking,
                })

        await sse.send("agent_done", {
            "agent": agent.id,
            "full_content": "".join(full_text),
        })
```

効果： - 初トークンが約 1 秒以内に到着（待機ゼロ化） - 5 体並列ゆえ画面 5 分割でリアルタイム表示

11.2 段階的開示 (F-011)

完了したエージェントから順次画面に固定表示する。最遅エージェントの遅延が体感に影響なくなる。

```
# 完了イベントを発火順に処理 (asyncio.as_completed)
async def round_with_progressive_disclosure(agents, prompt, sse):
    tasks = [stream_agent(a, prompt, sse) for a in agents]
    for completed in asyncio.as_completed(tasks):
        result = await completed
        # 完了直後に SSE で「読み込んでよい」シグナルを送る
        await sse.send("agent_ready_for_reading", {"agent": result.agent_id})
```

UI 側は `agent_ready_for_reading` を受信した時点で該当パネルを「読了可」状態に切り替え、社長が読み始める。

11.3 プロンプトキャッシュ (F-014)

Anthropic API の `cache_control` パラメータを使用する。固定部分（システムプロンプト・会社コンテキスト）を `ephemeral` キャッシュとしてマーク：

```
system = [
    {
        "type": "text",
        "text": AGENT_PROMPTS[agent.id],
        "cache_control": {"type": "ephemeral"},          # ← キャッシュ対象
    },
    {
        "type": "text",
        "text": company_context,
        "cache_control": {"type": "ephemeral"},
    },
    {
        "type": "text",
        "text": principle_iv_template,                  # 反論義務テンプレート
        "cache_control": {"type": "ephemeral"},
    },
]
```

効果： - 2回目以降のセッションで該当部分のコストが **90% 割引** - 応答開始も **約 2 倍速**（プロセス済みプレフィクスから開始） - TTL 5 分。連続セッションには有効、長時間放置後は再度キャッシュ作成

11.4 Extended Thinking 可視化 (F-012)

熟考モードのみで有効化。Claude の `thinking` パラメータを使い、思考過程を SSE 経由で画面に流す：

```

response = await client.messages.create(
    model="claude-opus-4-20250514",
    max_tokens=2048,
    thinking={
        "type": "enabled",
        "budget_tokens": 8192,
    },
    system=agent.system_prompt,
    messages=[{"role": "user", "content": prompt}],
)

```

社長は「ただ待つ」のではなく「エージェントの思考を覗き見る」体験を得る。これは体感時間を主観的に大きく短縮する。

11.5 バックグラウンド実行+通知 (F-013, N-001~N-003)

```

[POST /deliberations { background: true }]
    ↓
即座に 202 Accepted で session_id 返却
    ↓
熟議は background task で実行 (FastAPI BackgroundTasks)
    ↓
完了時:
  - Web Push API でブラウザ通知 (PWA Service Worker 経由)
  - 任意で SMTP メール送信
    ↓
通知 → 1 クリックで /deliberations/{session_id} 結果画面へ

```

熟考モード（5分）でも社長は他業務を継続でき、体感時間は事実上ゼロになる。

11.6 まとめ：体感時間の構造化削減

元の体感問題	解決策	効果
「3分間ブランク画面」	F-010 ストリーミング	初トークン 1 秒で「動き始める」
「最遅エージェント待ち」	F-011 段階的開示	完了順に読める
「2回目も同じ時間」	F-014 プロンプトキャッシュ	半減
「ただ待つだけ」	F-012 Extended Thinking	思考を覗き見る
「ずっと画面前で待機」	F-013 バックグラウンド	体感ゼロ

12. 変更履歴

バージョン	日付	内容
0.1	2026-05-26	骨格作成
0.2	2026-05-26	Opus顧問審査結果を反映。エージェント数 8→5 体。FastAPI+HTMX採用。スキーマ拡充。Pre-mortem・多様性チェック追加。
0.3	2026-05-26	CFIM白書 v1.2 との整合。§9 を「段階的検証（PRM 拡張）」として再構成。§9.2 に説得依存度検定（Liu ら 2026 対策）を追加。閾値 0.3 と白書 $\Phi < 0.15$ の対応を明記。
0.4	2026-05-26	速度最適化を全面導入。モード3段（早足/常足/熟考）追加。§3 config.yaml にモード・キャッシュ・ストリーミング・Extended Thinking・バックグラウンド設定を統合。§5.2/5.3 にモード別レイテンシ・コスト試算。§7.2/7.3 にストリーミング対応 SSE イベント追加。新規 §11「速度最適化設計」に実装パターンを明記。